

智能感知与自动驾驶技术

谭国平

gptan@hhu.edu.cn

河海大学计算机与软件学院

2025年8月

智能感知与自动驾驶技术

- 第1章 绪论
 - 第2章 视觉传感器
 - 第3章 自动驾驶车道线检测
 - 第4章 自动驾驶目标检测
 - 第5章 自动驾驶决策控制
-

任务目标：基于强化学习的自动驾驶决策与控制



第5章 基于强化学习的自动驾驶决策与控制

- 5.1 强化学习简介
 - 5.2 强化学习算法
 - 5.3 应用案例-自动驾驶决策控制
-

5.1 强化学习简介

本节主要介绍强化学习的基本概念和智能体核心组件等知识。

讨论主题：

强化学习特点
环境与智能体
奖励与回报
状态
策略
价值函数
模型

5.1.1 强化学习特点

“强化学习” 课程材料

1. 自动驾驶仿真器 Metadrive:

<https://github.com/metadriverse/metadrive>

2. 王树森: Deep Reinforcement Learning (中文视频教程)

<https://github.com/wangshusen/DRL>

3. Learn Deep Reinforcement Learning in 60 days! Lectures & Code in Python:

<https://github.com/andri27-ts/Reinforcement-Learning>

4. Deep Reinforcement Learning Nanodegree:

<https://github.com/udacity/deep-reinforcement-learning>

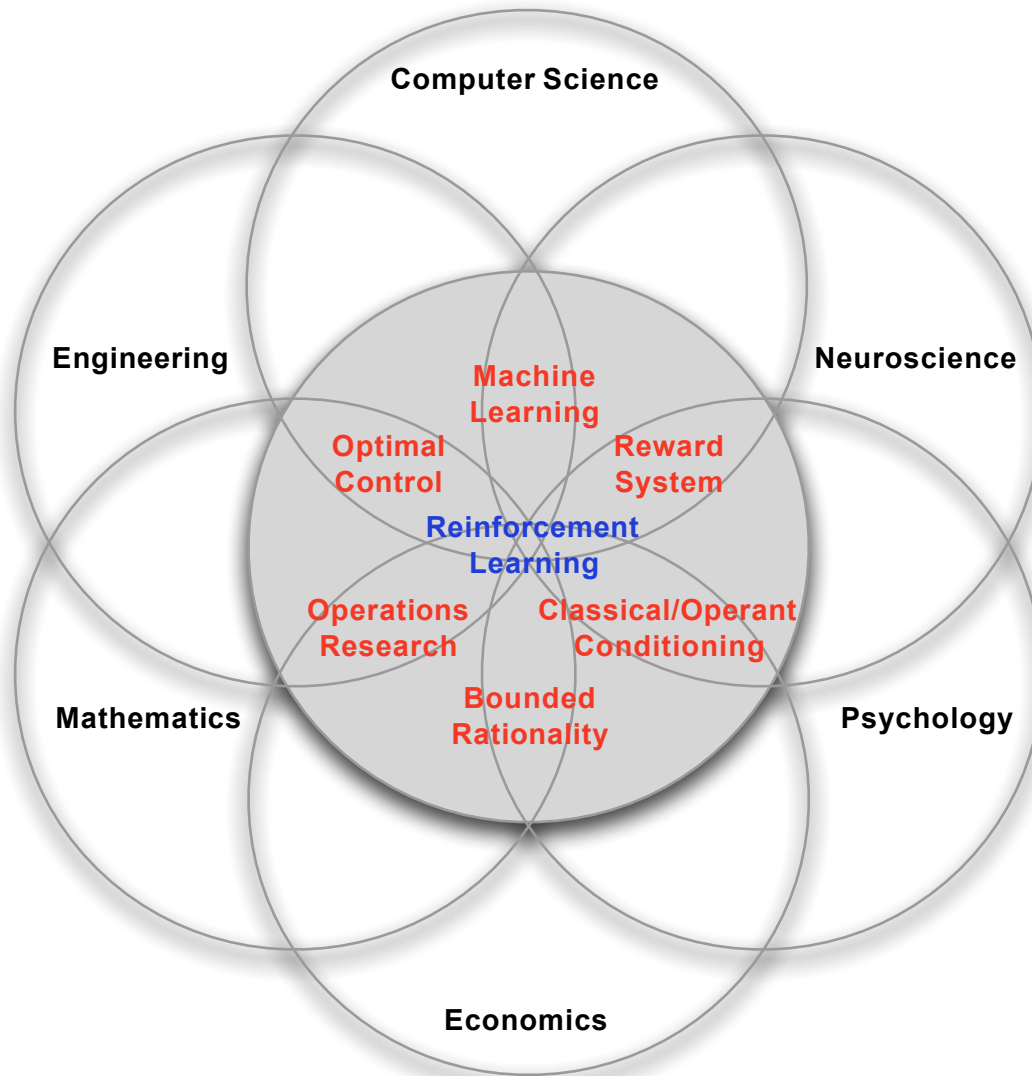
5. Reinforcement Learning: An Introduction:

<https://github.com/ShangtongZhang/reinforcement-learning-an-introduction>

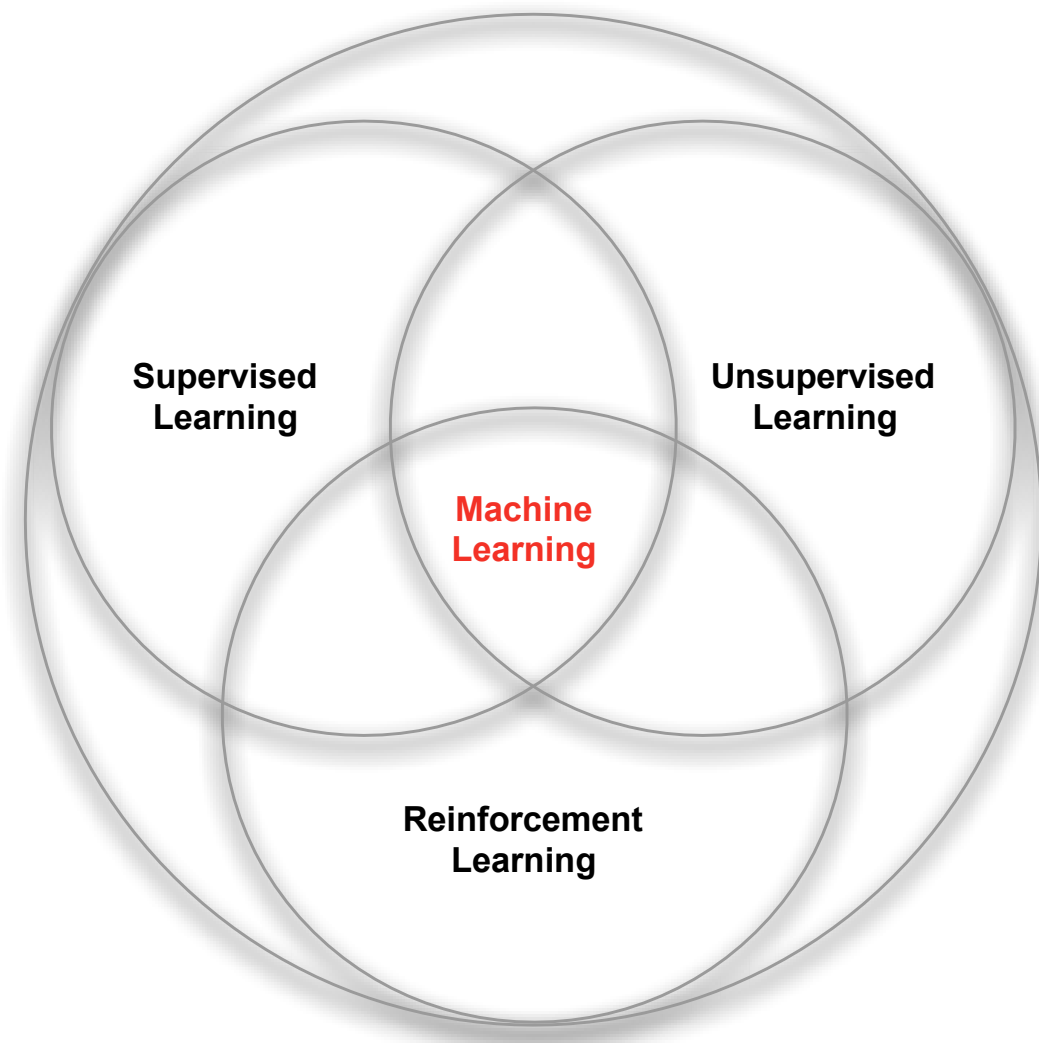
6. Implementation of Reinforcement Learning Algorithms.:

<https://github.com/dennybritz/reinforcement-learning>

学科交叉中的强化学习(Reinforcement Learning)



机器学习分支



强化学习特征

强化学习与其他机器学习范式的区别：

1. There is no supervisor, only a reward signal
没有标签或监督，只有环境给出的奖励值作为学习依据。
2. Feedback is delayed, not instantaneous
奖励常滞后，当前动作的好坏需经过多步才能体现。
3. Time really matters (sequential, non i.i.d data)
数据按时间顺序生成，前后相关，非独立同分布。
4. Agent's actions affect the subsequent data it receives
智能体的行为改变环境，进而影响后续获得的数据分布。

Rewards(奖励)

1. 奖励 R 是时刻 t 反馈的一个标量，反映该步动作好坏；
2. 智能体需最大化整条轨迹的累积奖励。
强化学习思想起源于奖励假说(reward hypothesis)

奖励假说(reward hypothesis)

所有目标都可归结为“最大化长期累积奖励的期望值”。

Rewards(奖励)的例子

1. 直升机飞行

1. 正向奖励: 按期望轨迹飞行
2. 负向奖励: 发生坠机

2. 棋类游戏

1. 正向奖励: 赢一局
2. 负向奖励: 输一局

3. 电站控制

1. 正向奖励: 成功发电
2. 负向奖励: 超出安全阈值

4. 机器人步行

1. 正向奖励: 向前移动
2. 负向奖励: 跌倒

5. 自动驾驶

1. 正向奖励: 按照期望轨迹行驶
2. 负向奖励: 发生碰撞

强化学习擅长序贯决策 (Sequential Decision Making)

1. 目标

选择一系列动作，使 **长期累积奖励最大化**，而非只看眼前回报。

2. 关键特性

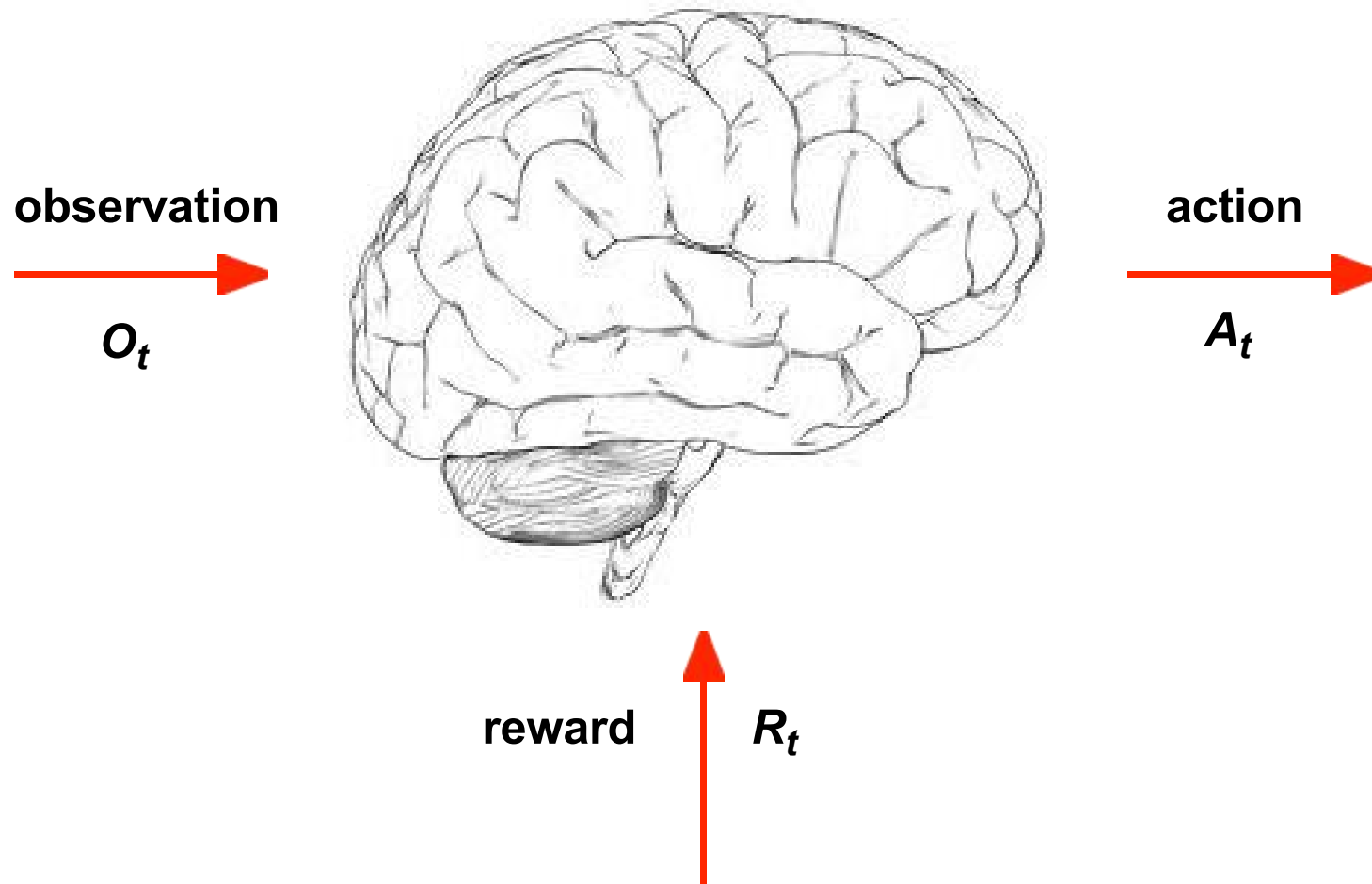
1. **动作具有长期影响**：当前决策的后果可能在很久之后才显现。
2. **奖励延迟**：回报不会立即到手。
3. **可能牺牲短期利益**：为了更大的长期收益，需要“忍一时”。

3. 具体例子

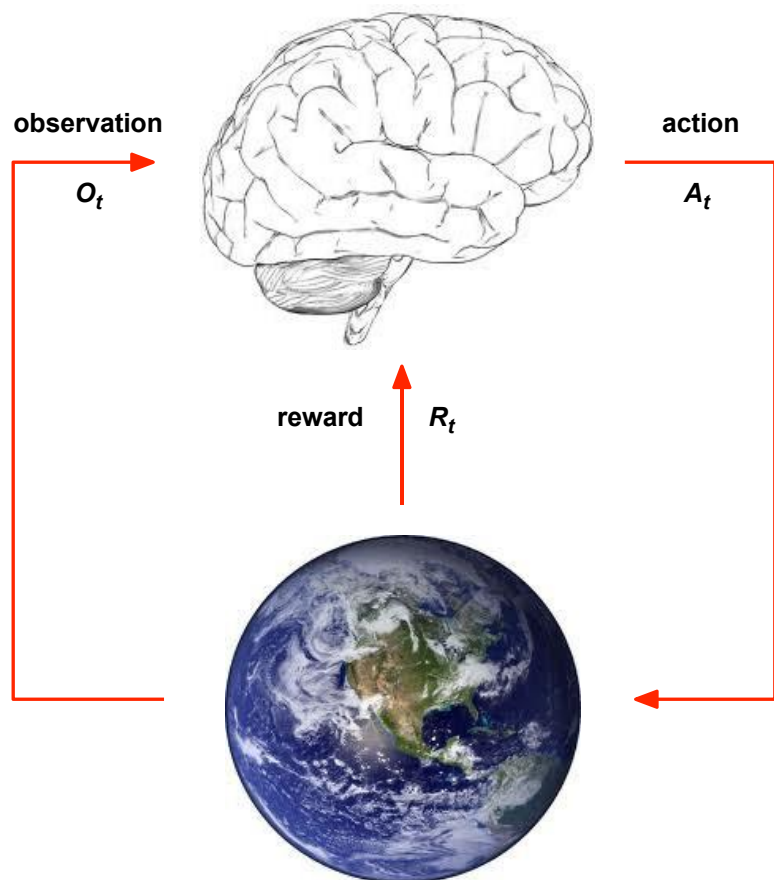
1. **金融投资**：今天买入，数月后才见效。
2. **给直升机加油**：当下花时间加油，可避免未来坠机。
3. **棋类对弈**：当下“弃子”看似吃亏，却能提升许多步后的胜率。
4. **自动驾驶**：当下看似保守的减速或绕行，可避免未来碰撞并换来更长远的行程安全与效率。

5.1.2 智能体与环境

智能体与环境 (Agent and Environment)



智能体与环境 (Agent and Environment)



每个时刻 t , 智能体与环境分别执行如下操作:

智能体:

- 执行动作 A_t
- 接收观测 O_t
- 接收标量奖励 R_t

环境:

- 接收动作 A_t
- 产生并发出下一观测 O_{t+1}
- 产生并发出下一奖励 R_{t+1}

历史与状态 (History and State)

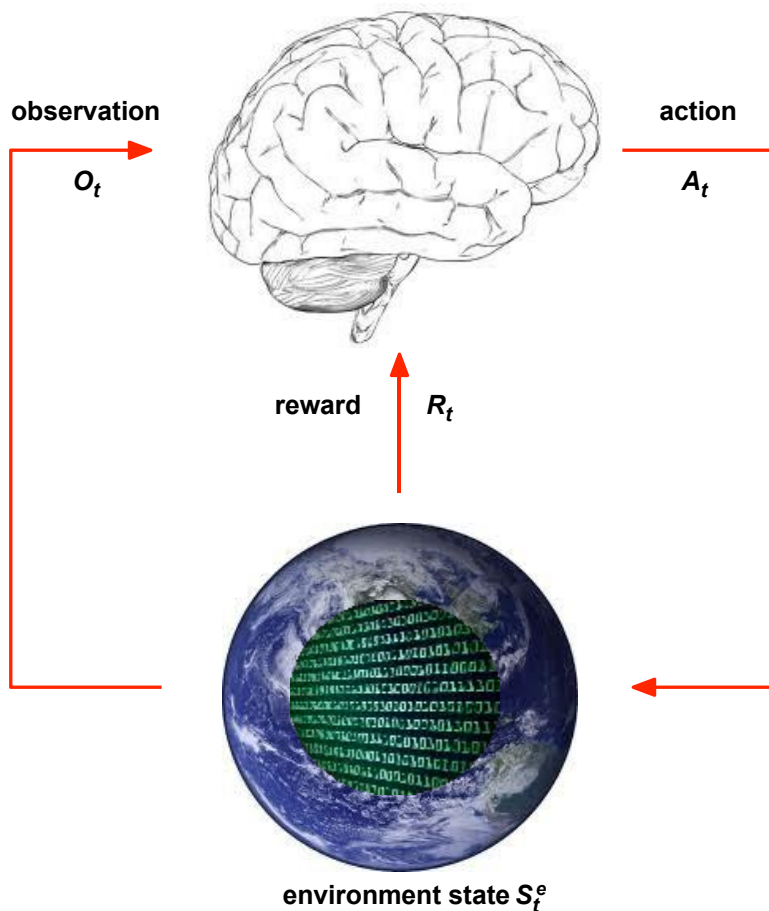
- The **history** is the sequence of observations, actions, rewards

$$H_t = O_1, R_1, A_1, \dots, A_{t-1}, O_t, R_t$$

- i.e. all observable variables up to time t
- i.e. the sensorimotor stream of a robot or embodied agent
- What happens next depends on the history:
 - The agent selects actions
 - The environment selects observations/rewards
- **State** is the information used to determine what happens
- next Formally, state is a function of the history:

$$S_t = f(H_t)$$

环境状态 (Environment State)



1. 定义

环境状态 S_t^e 是环境内部用来决定下一时刻观测与奖励的“私有表示”。

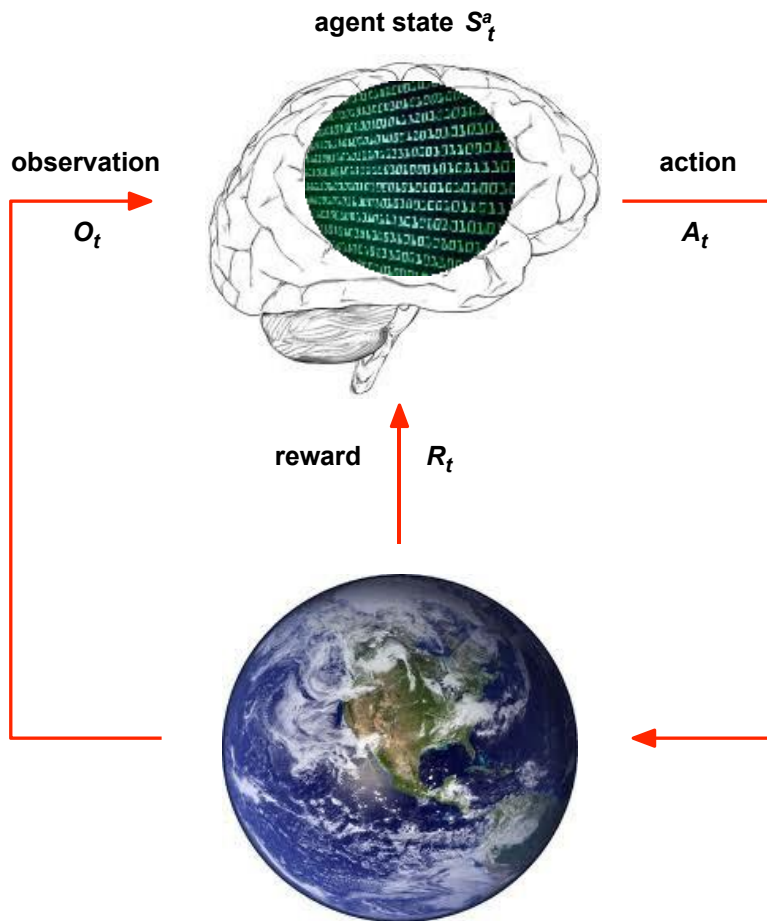
2. 对智能体不可见

智能体通常看不到完整的环境状态，只能拿到观测 O_t 。

3. 信息冗余

即使观测里包含了部分环境状态，也可能夹杂与任务无关的噪声或冗余信息。

智能体状态 (Agent State)



- **定义：** 智能体状态是其内部用来决定下一动作的信息集合。
- **作用：** 智能体状态是强化学习算法实际利用的“内部表示”，而非环境或观测本身。

形式上，智能体状态可以表示为“历史”的函数：

$$S_t^a = f(H_t)$$

马尔可夫状态 (Information State: Markov State)

An **information state** (a.k.a. **Markov state**) contains all useful information from the history.

Definition

A state S_t is **Markov** if and only if

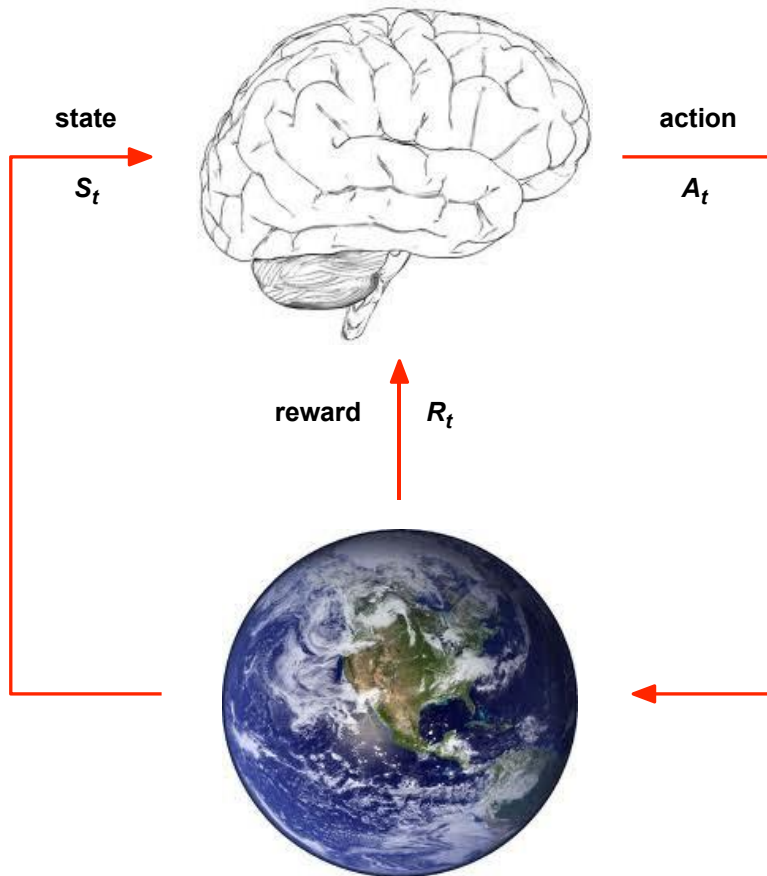
$$P[S_{t+1} \mid S_t] = P[S_{t+1} \mid S_1, \dots, S_t]$$

- “The future is independent of the past given the present”

$$H_{1:t} \rightarrow S_t \rightarrow H_{t+1:\infty}$$

- Once the state is known, the history may be thrown away
- i.e. The state is a sufficient statistic of the future
- The environment state S_t^e is Markov
- The history H_t is Markov

完全可观测环境 (Fully Observable Environments)



Full observability: agent **directly** observes environment state

$$O_t = S_t^a = S_t^e$$

- Agent state = environment state = information state
- Formally, this is a **Markov decision process** (MDP)

部分可观测环境 (Partially Observable Environments)

1. 部分可观测

智能体无法直接看到完整的环境状态，只能通过间接、有限的观测来推断世界。(例如机器人摄像头看不到绝对位置、扑克玩家只能看到公共牌)

2. 数学模型

形式化为部分可观测马尔可夫决策过程 (POMDP) 。

3. 构建智能体状态

由于观测不足，智能体必须自行构造内部状态 S_t^a ，常见做法：

完整历史轨迹： 把全部观测-动作-奖励历史轨迹序列当状态。

$$S_t^a = H_t$$

信念状态： 维护对环境真实状态的概率分布。

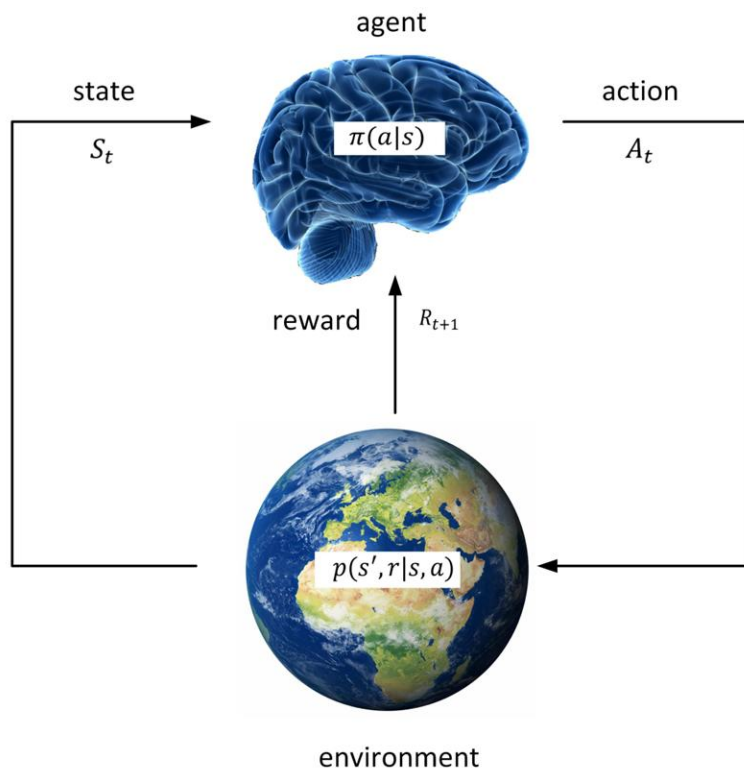
$$S_t^a = \{P[S_t^e = s^1], \dots, P[S_t^e = s^n]\}$$

循环神经网络： 用 RNN 把历史压缩成隐藏状态。

$$S_t^a = \sigma(S_{t-1}^a W_s + O_t W_o)$$

5.1.3 智能体核心组件

RL智能体主要组件(Major Components of RL Agent)



1.策略 (Policy)

智能体的行为函数，在给定状态下，决定采取何种动作。

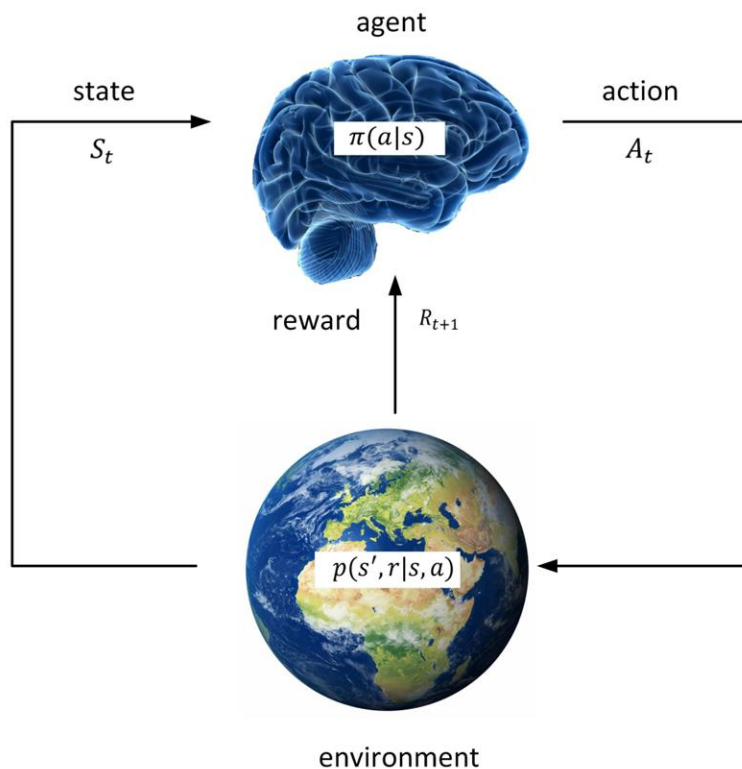
2.价值函数 (Value Function)

评估每个状态或状态-动作对的好坏程度，用于衡量长期收益的期望。

3.模型 (Model)

智能体对环境动态的内在表示，用于预测下一状态与奖励。

策略(Policy)



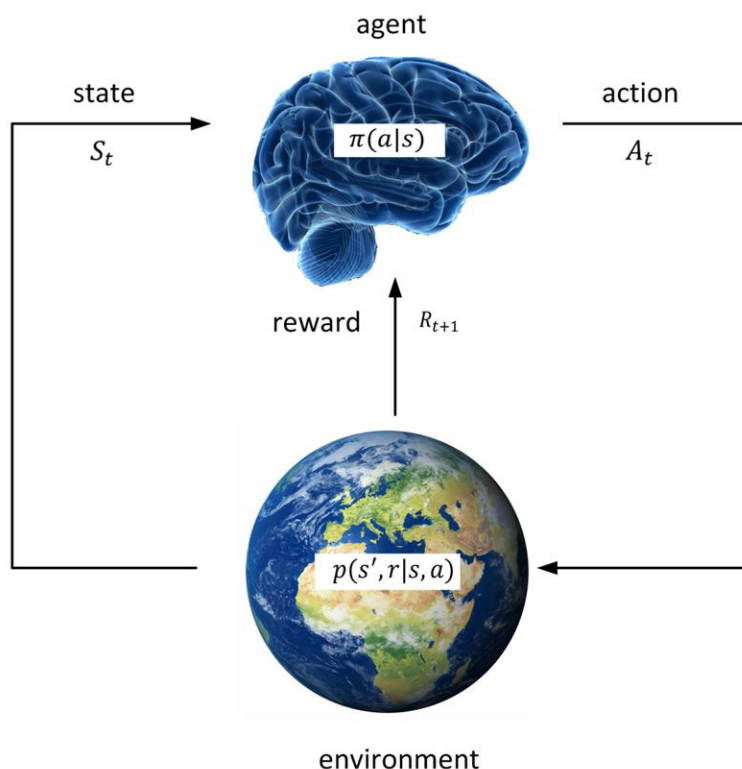
- **策略 $\pi(a|s)$** ：是智能体的行为方式，即从状态到动作的映射。

- **两种形式：**

- **确定性策略**：给定状态 s ，动作 a 由函数 $\pi(s)$ 唯一确定。

- **随机性策略**：给定状态 s ，按概率分布 $\pi(a|s)$ 随机选择动作， $\pi(a|s) = P[A_t=a | S_t=s]$ 。

价值函数(Value Function)



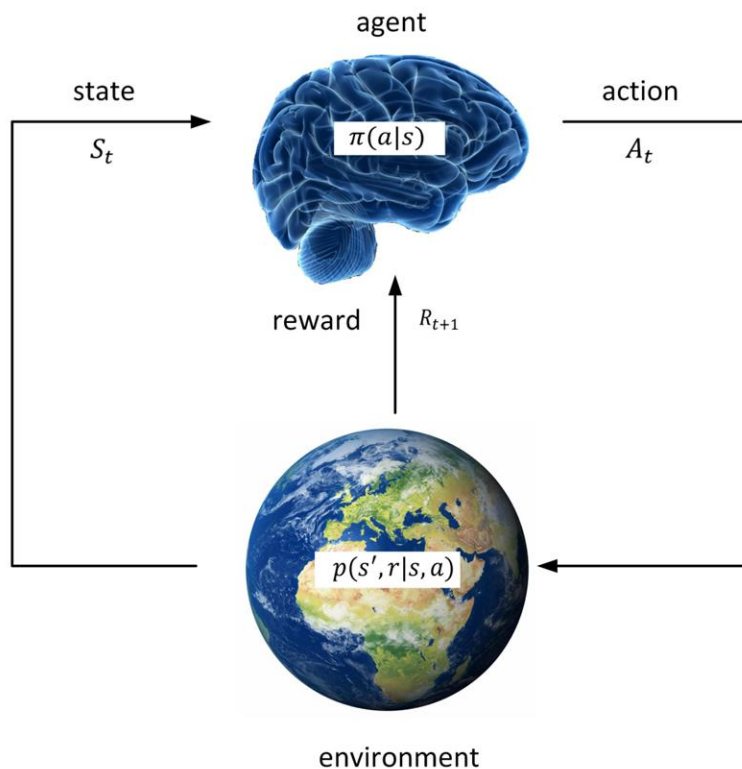
价值函数: 对未来奖励的预测，用来衡量一个状态 s 的“好坏”。即从状态 s 出发，按策略 π 行动所能获得的**折扣累积回报期望值**:

$$V_{\pi}(s) = E_{\pi} [\sum_{n=1}^{\infty} \gamma^{n-1} r_{t+n} | S_t = s]$$

$\gamma \in [0,1]$ 是折扣率。当 γ 接近于 0 时，智能体更在意短期回报；而当 γ 接近于 1 时，长期回报变得更重要。

用途: 智能体通过比较状态在不同策略下的价值，选择更优策略。

模型(Model)



功能：对环境下一步行为的预测器。

两个核心输出：

1. 给出状态转移概率：

$$P(s' | s, a) = P[S_{t+1} = s' | S_t = s, A_t = a]$$

2. 给出即时奖励的期望：

$$R(s, a) = E[R_{t+1} | S_t = s, A_t = a]$$

5.2 强化学习算法

本节主要介绍强化学习算法主流分类方法，重点介绍在自动驾驶领域常用的DQN和Actor-Critic深度强化学习方法。

讨论主题：

强化学习算法分类

DQN算法

Actor-Critic算法

5.2.1 强化学习算法分类

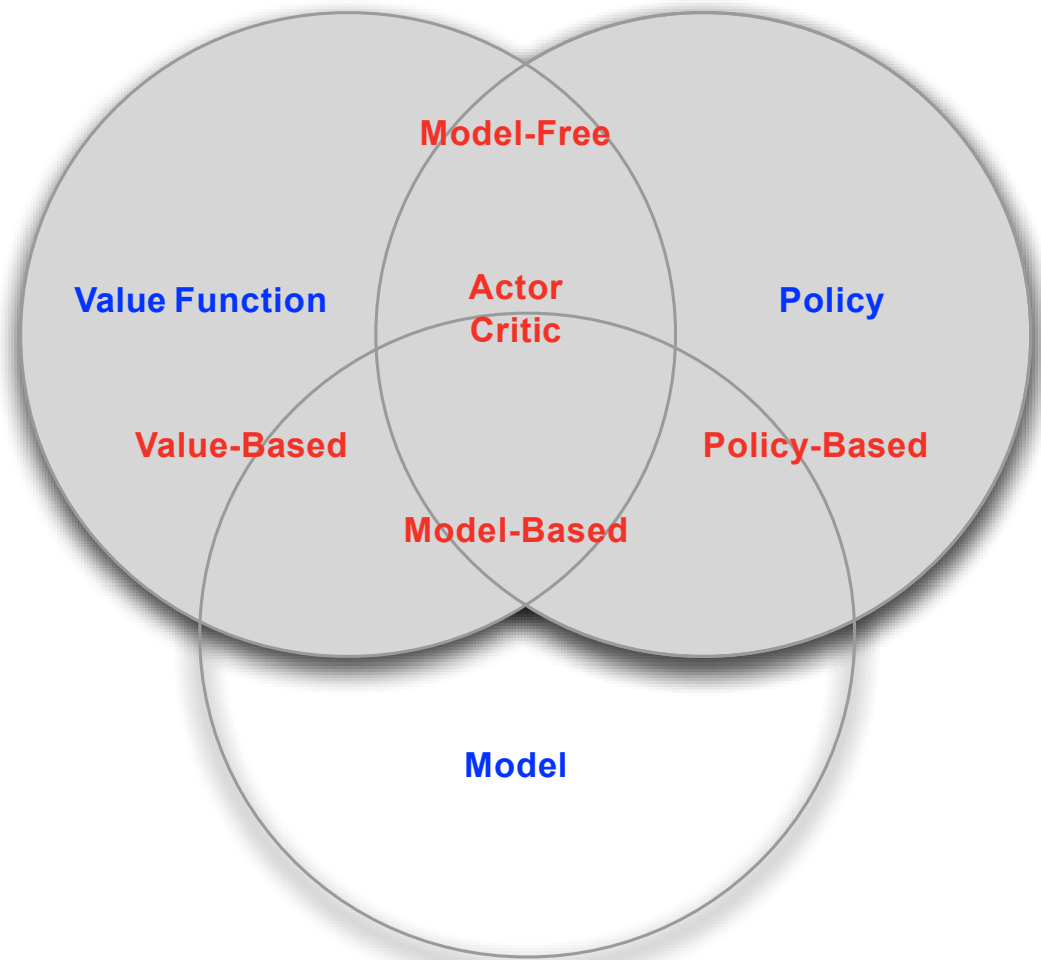
强化学习算法分类 - I

- Value Based
 - No Policy (Implicit)
 - Value Function
 - Policy Based
 - Policy
 - No Value Function
 - Actor Critic
 - Policy
 - Value Function
-

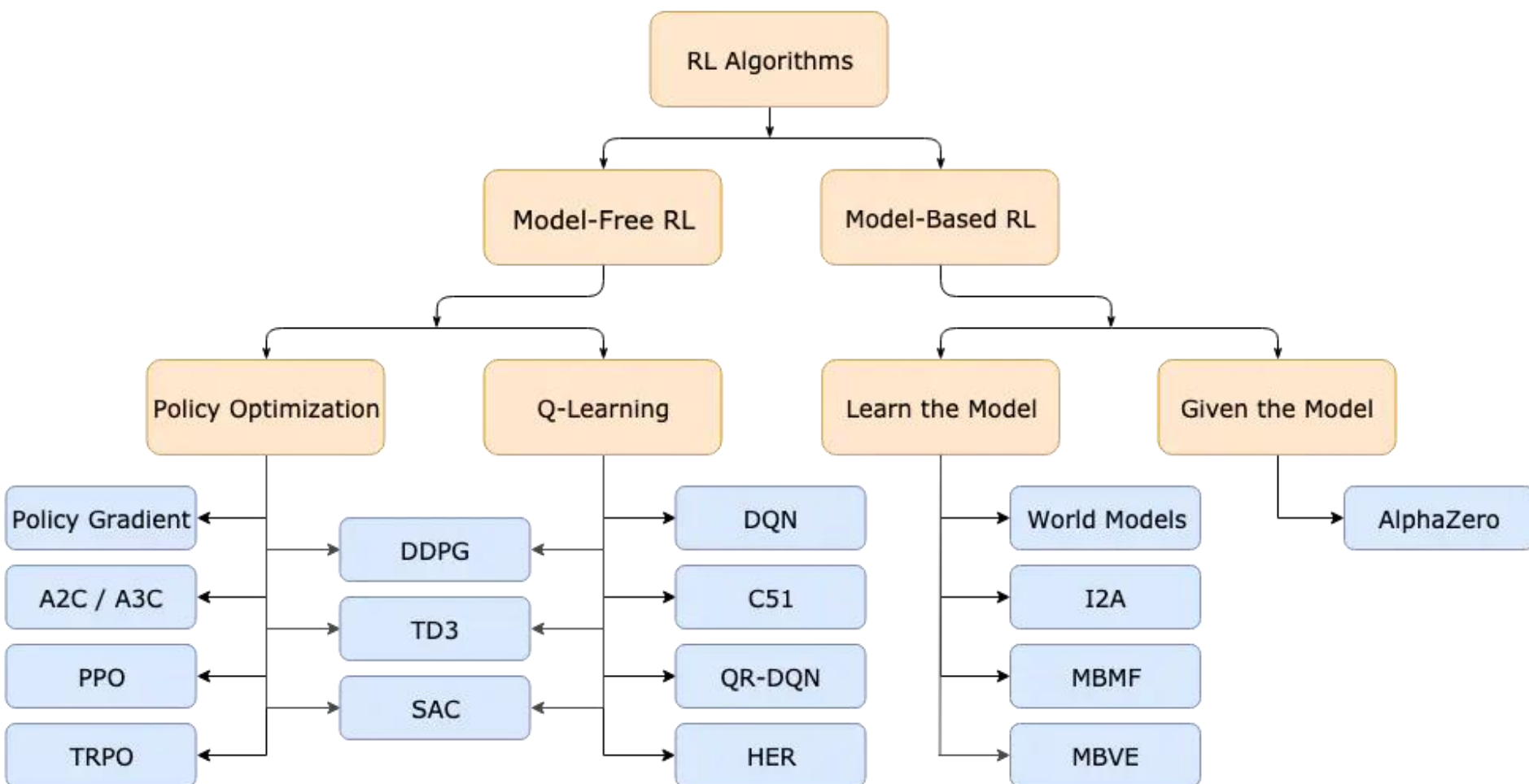
强化学习算法分类 - II

- Model Free
 - Policy and/or Value Function
 - No Model
 - Model Based
 - Policy and/or Value Function
 - Model
-

强化学习算法各类型之间的关联性



经典强化学习算法分类(OpenAI)



5.2.2 DQN算法

如何评估策略 $\pi_{\theta}(a|s)$?

- 两个价值函数
 - “状态” 价值函数
 - “状态-动作对” 价值函数

状态价值函数

- 一个策略 π 期望回报可以分解为

$$\begin{aligned}\mathbb{E}_{\tau \sim p(\tau)}[G(\tau)] &= \mathbb{E}_{s \sim p(s_0)} \left[\mathbb{E}_{\tau \sim p(\tau)} \left[\sum_{t=0}^{T-1} \gamma^t r_{t+1} \mid \tau_{s_0} = s \right] \right] \\ &= \mathbb{E}_{s \sim p(s_0)} [V^\pi(s)],\end{aligned}$$

- 执行策略 π 的状态价值函数：从状态 s 开始，执行策略 π 得到的期望总回报

$$V^\pi(s) = \mathbb{E}_{\tau \sim p(\tau)} \left[\sum_{t=0}^{T-1} \gamma^t r_{t+1} \mid \tau_{s_0} = s \right]$$

Bellman方程

$$\begin{aligned} V^\pi(s) &= \mathbb{E}_{\tau_{0:T} \sim p(\tau)} \left[r_1 + \gamma \sum_{t=1}^{T-1} \gamma^{t-1} r_{t+1} \mid \tau_{s_0} = s \right] \\ &= \mathbb{E}_{a \sim \pi(a|s)} \mathbb{E}_{s' \sim p(s'|s,a)} \mathbb{E}_{\tau_{1:T} \sim p(\tau)} \left[r(s, a, s') + \gamma \sum_{t=1}^{T-1} \gamma^{t-1} r_{t+1} \mid \tau_{s_1} = s' \right] \\ &= \mathbb{E}_{a \sim \pi(a|s)} \mathbb{E}_{s' \sim p(s'|s,a)} \left[r(s, a, s') + \gamma \mathbb{E}_{\tau_{1:T} \sim p(\tau)} \left[\sum_{t=1}^{T-1} \gamma^{t-1} r_{t+1} \mid \tau_{s_1} = s' \right] \right] \\ &= \mathbb{E}_{a \sim \pi(a|s)} \mathbb{E}_{s' \sim p(s'|s,a)} \left[r(s, a, s') + \gamma V^\pi(s') \right]. \end{aligned}$$

“状态-动作对” 价值函数

- 状态-动作值函数是指初始状态为 s 并进行动作 a , 然后执行策略 π 得到的期望总回报。

$$Q^{\pi}(s, a) = \mathbb{E}_{s' \sim p(s'|s, a)} [r(s, a, s') + \gamma V^{\pi}(s')]$$

最优策略

- 最优策略：存在一个最优的策略 π^* ，其在所有状态上的期望回报最大。

$$\forall s, \quad \pi^* = \arg \max_{\pi} V^{\pi}(s).$$

- 难以实现
- 策略改进：
 - 值函数可以看作是对策略 π 的评估。
 - 如果在状态 s ，有一个动作 a 使得 $Q^{\pi}(s,a) > V^{\pi}(s)$ ，说明执行动作 a 比当前的策略 $\pi(a|s)$ 要好，我们就可以调整参数使得策略 $\pi(a|s)$ 的概率增加。

如何计算价值函数？

- 基于模型的强化学习算法
 - 基于MDP过程：状态转移概率 $p(s' | s, a)$ 和奖励函数 $R(s, a, s')$
 - 策略迭代
 - 值迭代
- 模型无关的强化学习
 - 无MDP过程
 - 蒙特卡罗采样方法
 - 时序差分学习

经典强化学习算法 – Q Learning

算法 15.4: Q 学习算法

输入: 状态空间 $\mathcal{S}$, 动作空间 $\mathcal{A}$, 折扣率 γ , 学习率 α

- 1 随机初始化 $Q(s, a)$;
 - 2 $\forall s, \forall a, \pi(a|s) = \frac{1}{|\mathcal{A}|}$;
 - 3 **repeat**
 - 4 初始化起始状态 s ;
 - 5 **repeat**
 - 6 在状态 s , 选择动作 $a = \pi^\epsilon(s)$;
 - 7 执行动作 a , 得到即时奖励 r 和新状态 s' ;
 - 8 $Q(s, a) \leftarrow Q(s, a) + \alpha \left(r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right)$;
 - 9 $s \leftarrow s'$;
 - 10 **until** s 为终止状态;
 - 11 **until** $\forall s, a, Q(s, a)$ 收敛;
- 输出: 策略 $\pi(s) = \arg \max_{a \in |\mathcal{A}|} Q(s, a)$
-

最优“状态-动作对”价值函数的近似

Approximate the Q Function

Goal: Win the game ($\approx$ maximize the total reward.)

Question: If we know $Q^*(s, a)$, what is the best action?

- Obviously, the best action is $a^* = \underset{a}{\operatorname{argmax}} Q^*(s, a)$.

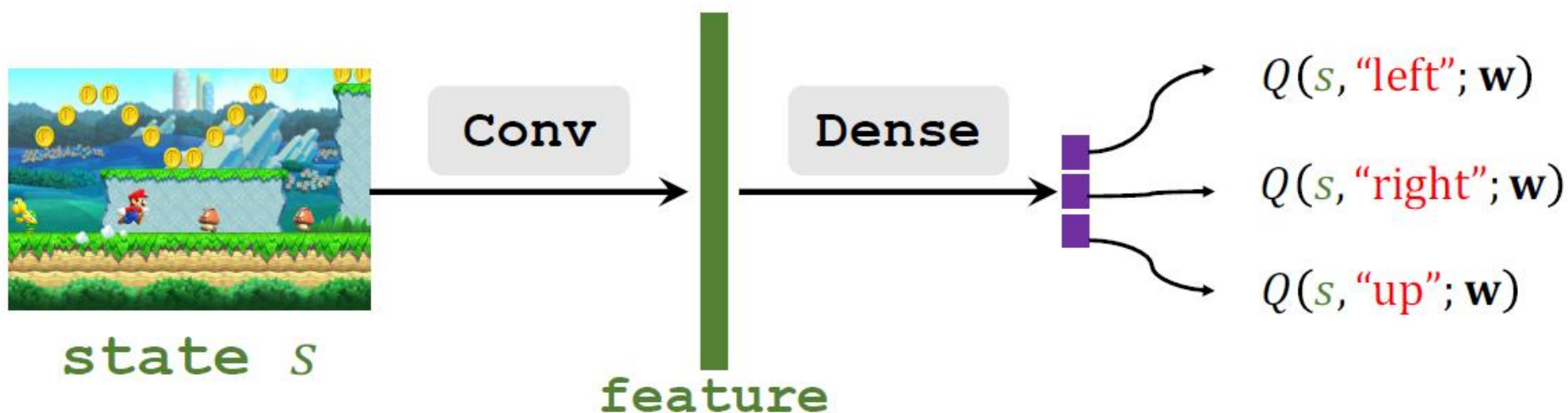
Challenge: We do not know $Q^*(s, a)$.

- Solution: Deep Q Network (DQN)
- Use neural network $Q(s, a; \mathbf{w})$ to approximate $Q^*(s, a)$.

Deep Q Network 网络架构

Deep Q Network (DQN)

- Input shape: size of the screenshot.
- Output shape: dimension of action space.



DQN的时序差分学习算法

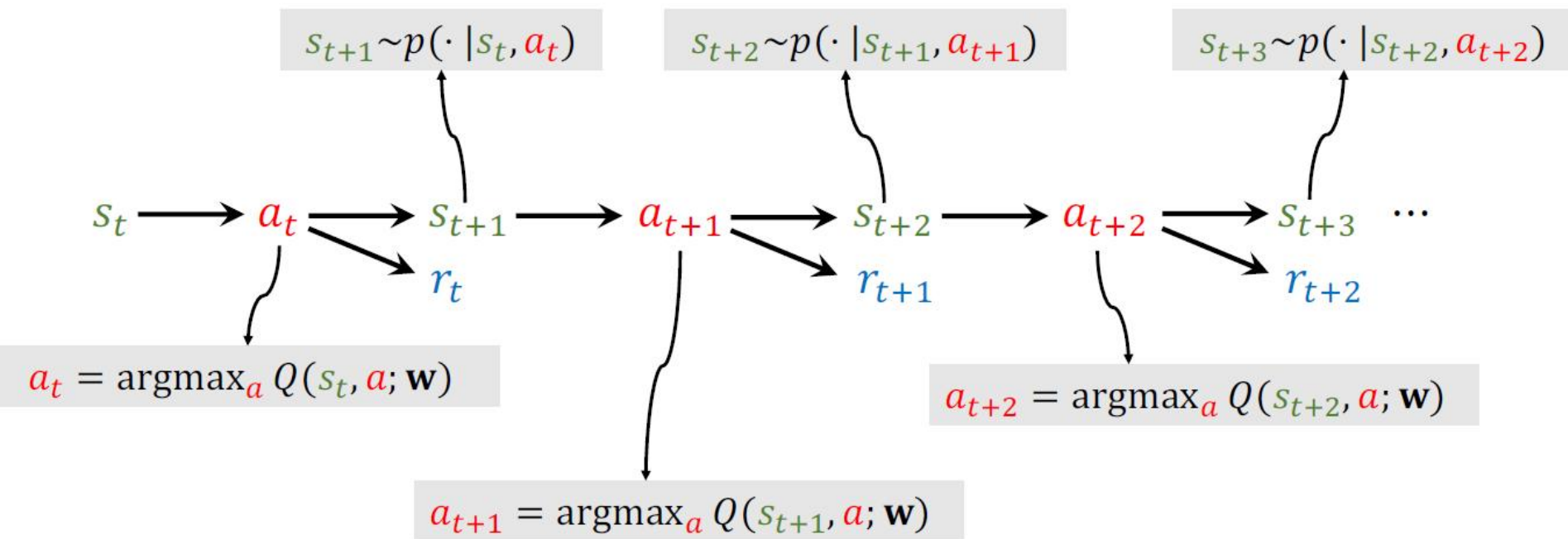
Temporal Difference (TD) Learning

Algorithm: One iteration of TD learning.

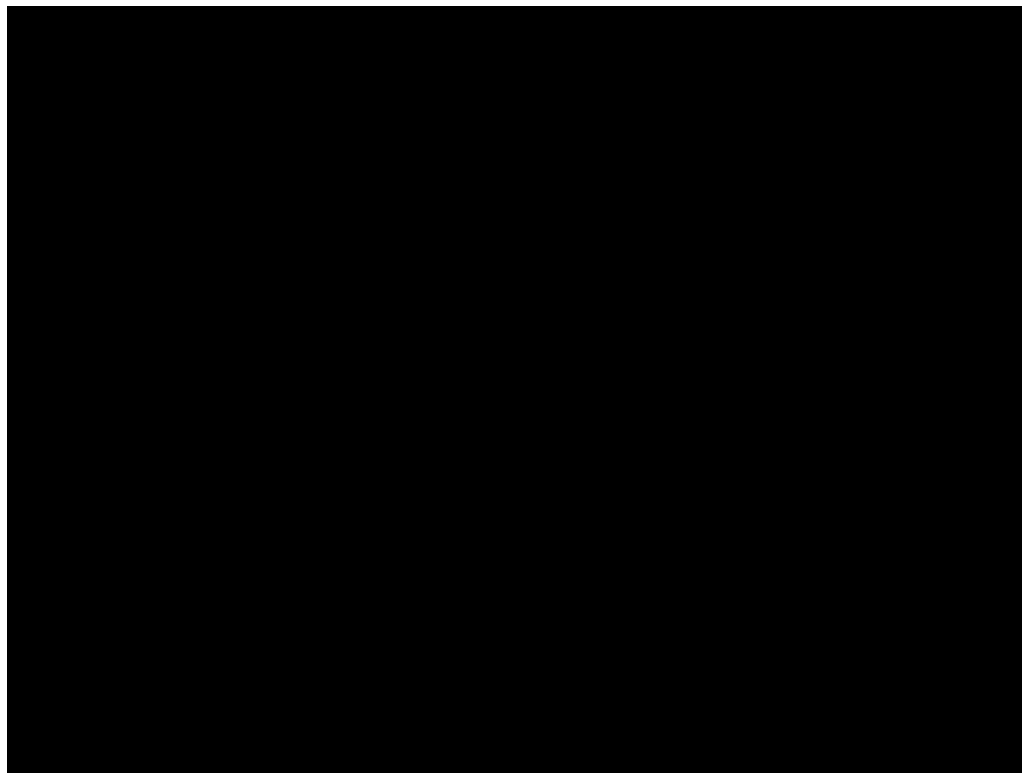
1. Observe state $S_t = s_t$ and perform action $A_t = a_t$.
2. Predict the value: $q_t = Q(s_t, a_t; \mathbf{w}_t)$.
3. Differentiate the value network: $\mathbf{d}_t = \frac{\partial Q(s_t, a_t; \mathbf{w})}{\partial \mathbf{w}} \Big|_{\mathbf{w}=\mathbf{w}_t}$.
4. Environment provides new state s_{t+1} and reward r_t .
5. Compute TD target: $y_t = r_t + \gamma \cdot \max_a Q(s_{t+1}, a; \mathbf{w}_t)$.
6. Gradient descent: $\mathbf{w}_{t+1} = \mathbf{w}_t - \alpha \cdot (q_t - y_t) \cdot \mathbf{d}_t$.

DQN应用过程示例

Apply DQN to Play Game



DQN智能体玩游戏经典示例(Nature 2015)



Source(Google Deepmind): Mnih, V., Kavukcuoglu, K., Silver, D. et al. Human-level control through deep reinforcement learning. *Nature* 518, 529–533 (2015).

~~<https://doi.org/10.1038/nature14236>~~

5.2.3 Actor-Critic算法

Actor-Critic Method

policy network (actor)



value network (critic)



策略网络(Policy Network) 与 价值网络(Value Network)

Policy Network and Value Network

Definition: State-value function.

- $V_{\pi}(s) = \sum_a \pi(a|s) \cdot Q_{\pi}(s, a).$

Definition: function approximation using neural networks.

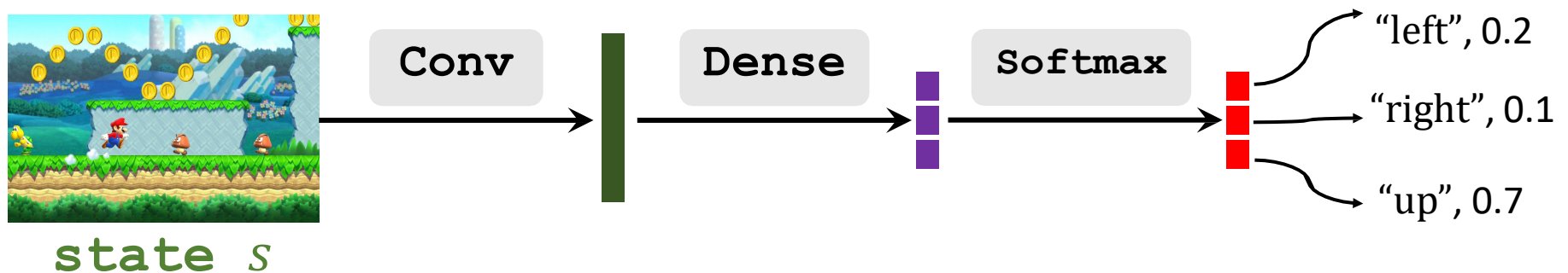
- Approximate policy function $\pi(a|s)$ by $\pi(a|s; \theta)$ (actor).
- Approximate value function $Q_{\pi}(s, a)$ by $q(s, a; w)$ (critic).

别称：演员网络(Actor Network) 与 评论网络(Critic Network)

策略网络(Actor)示例

Policy Network (Actor): $\pi(a|s; \theta)$

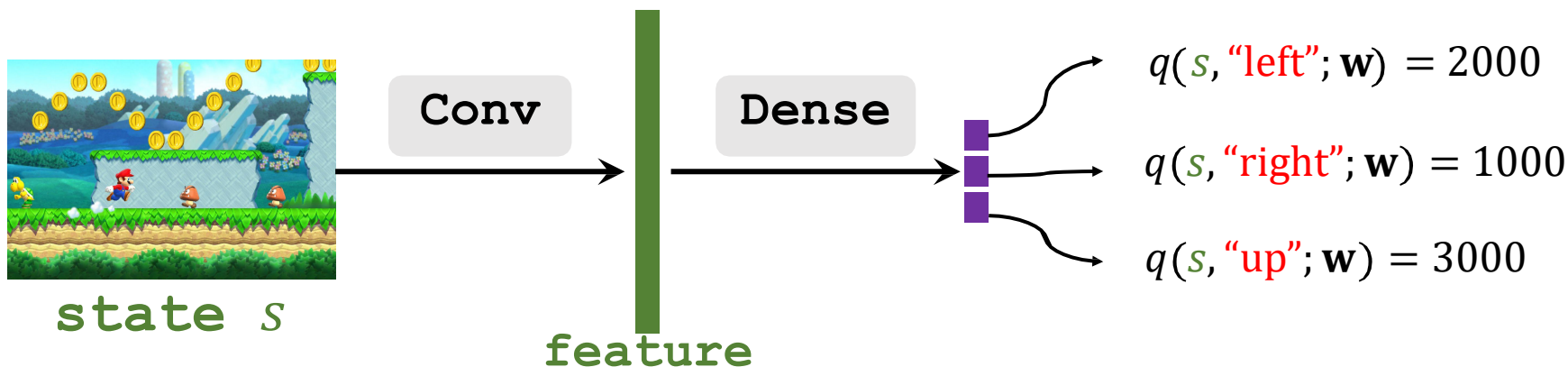
- Input: **state** s , e.g., a screenshot of Super Mario.
- Output: probability distribution over the **actions**.
- Let $\mathcal{A}$ be the set all actions, e.g., $\mathcal{A} = \{\text{"left"}, \text{"right"}, \text{"up"}\}$.
- $\sum_{a \in \mathcal{A}} \pi(a|s; \theta) = 1$. (That is why we use softmax activation.)



价值网络(Critic)示例

Value Network (Critic): $q(s, a; \mathbf{w})$

- Inputs: state s .
- Output: action-values of all the actions.



Actor 网络与Critic网络的训练方法

Training

Update the **policy network (actor)** by **policy gradient**.

- Seek to increase state-value: $V(\mathbf{s}; \boldsymbol{\theta}, \mathbf{w}) = \sum_a \pi(a|\mathbf{s}; \boldsymbol{\theta}) \cdot q(\mathbf{s}, a; \mathbf{w})$.
- Compute policy gradient: $\frac{\partial V(\mathbf{s}; \boldsymbol{\theta})}{\partial \boldsymbol{\theta}} = \mathbb{E}_A \left[\frac{\partial \log \pi(A|\mathbf{s}, \boldsymbol{\theta})}{\partial \boldsymbol{\theta}} \cdot q(\mathbf{s}, A; \mathbf{w}) \right]$.
- Perform gradient ascent.

Update the **value network (critic)** by **TD learning**.

- Predicted action-value: $q_t = q(\mathbf{s}_t, \mathbf{a}_t; \mathbf{w})$.
- TD target: $y_t = r_t + \gamma \cdot q(\mathbf{s}_{t+1}, \mathbf{a}_{t+1}; \mathbf{w})$
- Gradient: $\frac{\partial (q_t - y_t)^2 / 2}{\partial \mathbf{w}} = (q_t - y_t) \cdot \frac{\partial q(\mathbf{s}_t, \mathbf{a}_t; \mathbf{w})}{\partial \mathbf{w}}$.
- Perform gradient descent.

Actor-Critic算法流程

1. Observe state s_t and randomly sample $a_t \sim \pi(\cdot | s_t; \theta_t)$.
2. Perform a_t ; then environment gives new state s_{t+1} and reward r_t .
3. Randomly sample $\tilde{a}_{t+1} \sim \pi(\cdot | s_{t+1}; \theta_t)$. (Do not perform $\tilde{a}_{t+1}$!)
4. Evaluate value network: $q_t = q(s_t, a_t; \mathbf{w}_t)$ and $q_{t+1} = q(s_{t+1}, \tilde{a}_{t+1}; \mathbf{w}_t)$.
5. Compute TD error: $\delta_t = q_t - (r_t + \gamma \cdot q_{t+1})$.
6. Differentiate value network: $\mathbf{d}_{w,t} = \frac{\partial q(s_t, a_t; \mathbf{w})}{\partial \mathbf{w}} \Big|_{\mathbf{w}=\mathbf{w}_t}$.
7. Update value network: $\mathbf{w}_{t+1} = \mathbf{w}_t - \alpha \cdot \delta_t \cdot \mathbf{d}_{w,t}$.
8. Differentiate policy network: $\mathbf{d}_{\theta,t} = \frac{\partial \log \pi(a_t | s_t, \theta)}{\partial \theta} \Big|_{\theta=\theta_t}$.
9. Update policy network: $\theta_{t+1} = \theta_t + \beta \cdot q_t \cdot \mathbf{d}_{\theta,t}$.

Actor网络和Critic网络的作用

Roles of Actor and Critic

During training

- Agent is controlled by policy network (actor): $a_t \sim \pi(\cdot | s_t; \theta)$.
- Value network q (critic) provides the actor with supervision.

After training

- Agent is controlled by policy network (actor): $a_t \sim \pi(\cdot | s_t; \theta)$.
 - Value network q (critic) will not be used.
-

5.3 应用案例 - 自动驾驶决策控制

本节主要介绍基于Actor-Critic算法的自动驾驶决策控制实现方法。

讨论主题：

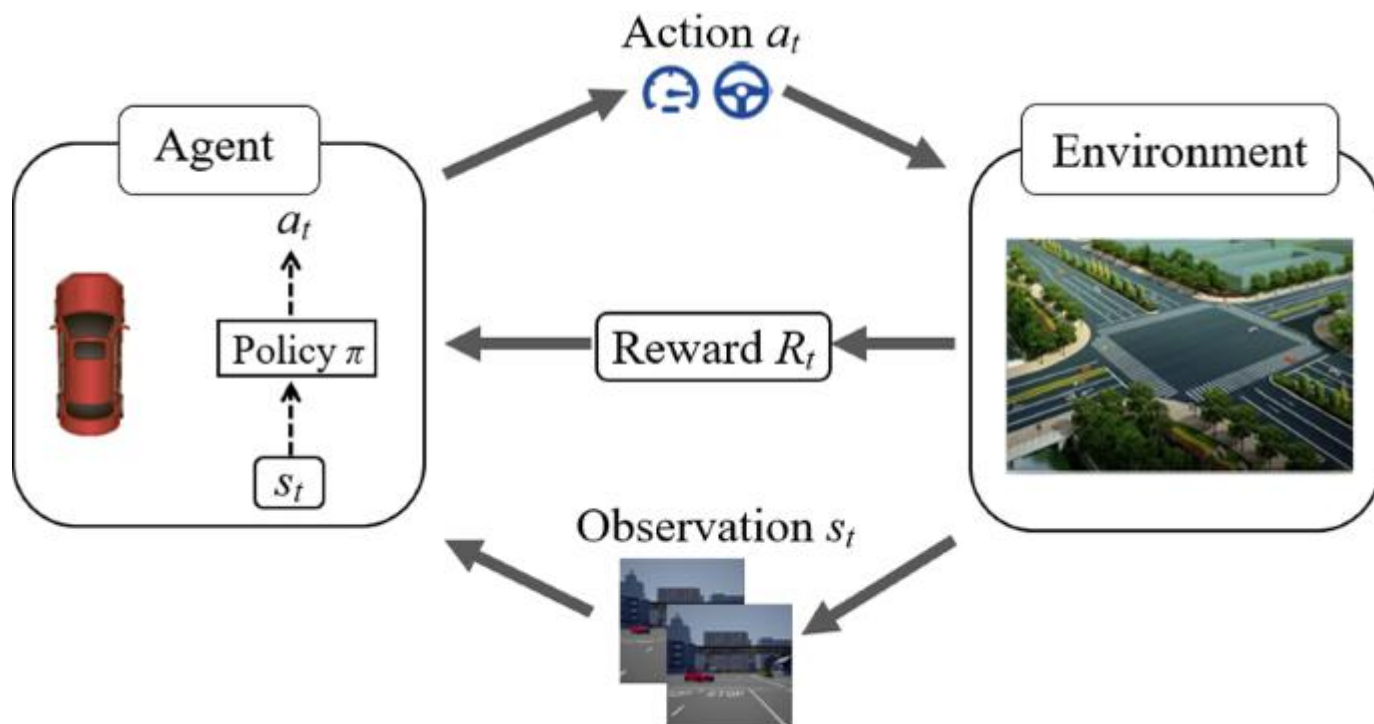
自动驾驶强化学习建模方法

PPO Actor-Critic网络设计

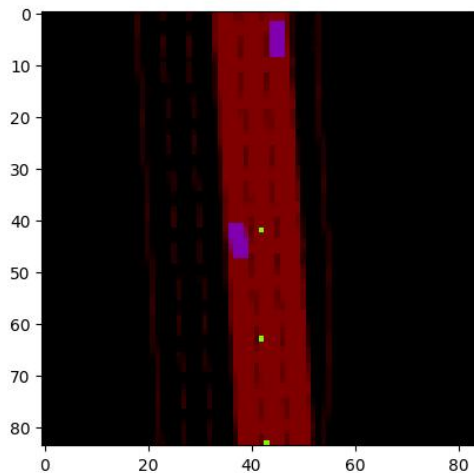
PPO算法训练

PPO智能体决策控制效果

自动驾驶决策控制 – 强化学习建模



强化学习建模—状态空间

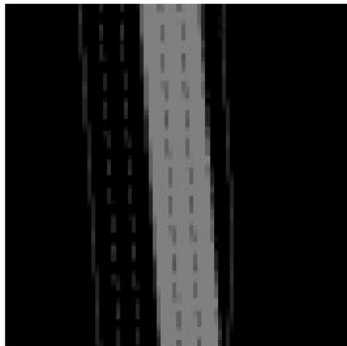


状态空间“鸟瞰图”

状态空间： 84×84 像素、覆盖 $42\text{ m} \times 42\text{ m}$ 的俯视图，用 5 张二值/灰度图层分别标记「路网与导航、邻车位置变化、自车位置变化」——训练PPO自动驾驶。输入智能体的图每个像素不是 RGB 三原色，而是 5 张二值灰度「图层」。如下所示：

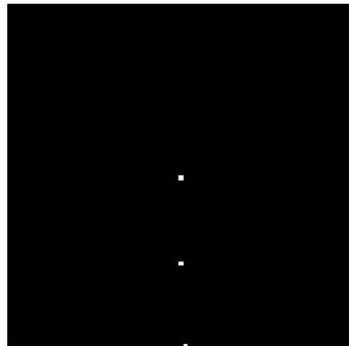
Road and navigation

Channel 0



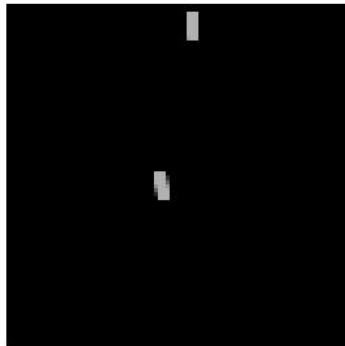
Ego now and previous pos

Channel 1



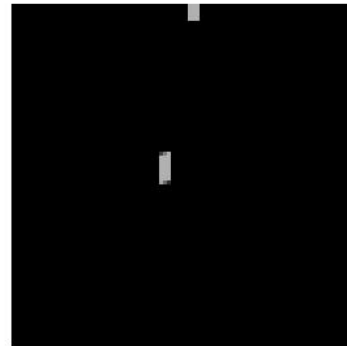
Neighbors at step t

Channel 2



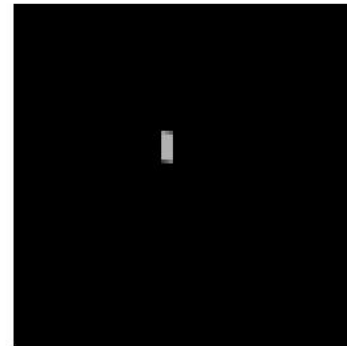
Neighbors at step t-1

Channel 3



Neighbors at step t-2

Channel 4



强化学习建模 – 动作空间



$$A = \{ (a1, a2) \mid -1 \leq a1 \leq 1, -1 \leq a2 \leq 1 \}$$

a1(steering): 方向盘角度

-1 左满舵, 0 回正, +1 右满舵

a2(throttle/brake): 油门/刹车合一量

+1 全油门, 0 松油门, -1 全刹车

原始动作	物理意义	控制量转换公式
a1	转向角	$\text{steering} = \text{max_steer} * a1$
a2 > 0	油门	$\text{engine_force} = \text{max_force} * a2$
a2 < 0	刹车	$\text{brake_force} = -\text{max_brake} * a2$

强化学习建模 – 奖励函数

$$r_t = \underbrace{\alpha \Delta s_t \cdot L_t \cdot \sigma_t}_{\text{推进}} + \underbrace{\beta \frac{v_t}{v_{\max}} \cdot \sigma_t}_{\text{速度}} + \underbrace{R_{\text{succ}} \cdot 1_{\text{goal}}}_{\text{成功}} + \underbrace{R_{\text{pen}} \cdot 1_{\text{fail}}}_{\text{失败}}$$

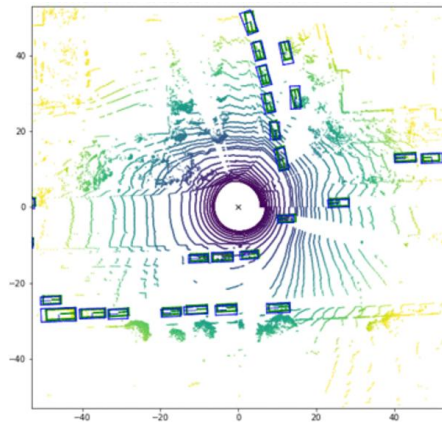
符号	含义	取值或计算	作用
Δs_t	一帧内沿车道前进距离	$\Delta s_t > 0$ 得正奖励, $\Delta s_t < 0$ 得负奖励	鼓励前进、惩罚倒退
L_t	车道居中系数	1 完全居中, 0 越线	抑制蛇形
σ_t	方向因子	+1 沿导航方向, -1 逆行	逆行时 <u>推进</u> /速度奖励变负
$\beta \frac{v_t}{v_{\max}}$	速度奖励	与车速成比例, σ_t 决定正负	鼓励合理加速
R_{succ}	成功奖励	到达终点一次性 +10	强化完成目标
R_{pen}	失败惩罚	越线、撞车、撞物、撞人行道一次性 -5	强化安全驾驶

自动驾驶：策略网络(Actor)示例

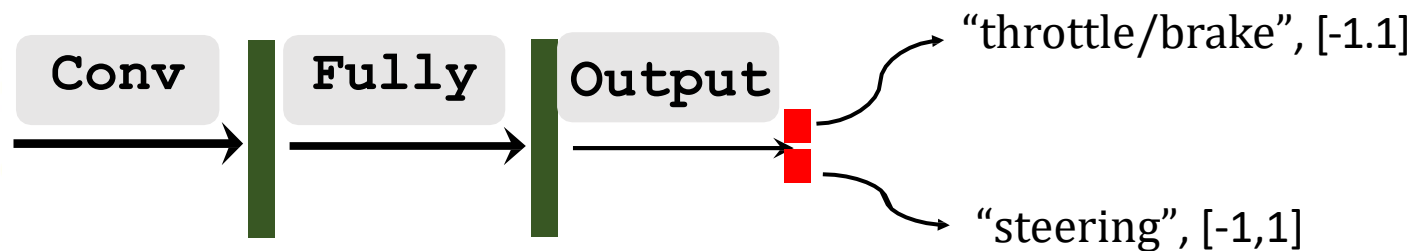
Policy Network (Actor): $\pi(a|s; \theta)$

- Input: **state** s , i.e., a snapshot of Bird's eyes' view.
- Output: probability distribution over the **actions**.
- Let $\mathcal{A}$ be the set all actions, i.e., $\mathcal{A} = \{\text{"throttle/brake"}, \text{"steering"}\}$.

state s



84×84×5 多通道
鸟瞰图像



自动驾驶PPO算法 – Actor网络设计

```
class Actor(nn.Module):
    def __init__(self):
        super().__init__()
        # input is size 84x84x5
        # output is size 2
        # 84x84x5 -> 20x20x16
        self.conv1 = nn.Conv2d(5, 16,
kernel_size=8, stride=4)
        # 20x20x16 -> 9x9x32
        self.conv2 = nn.Conv2d(16, 32,
kernel_size=4, stride=2)
        self.fc1 = nn.Linear(9*9*32, 256) # 9x9x32
-> 256
        self.fc2 = nn.Linear(256, 2) # 256 -> 2

    def forward(self, x: torch.Tensor) ->
torch.distributions.MultivariateNormal:
    x = F.relu(self.conv1(x))
    x = F.relu(self.conv2(x))
    x = torch.flatten(x, start_dim=1)
    x = F.relu(self.fc1(x))
    mu = self.fc2(x)
    # sigma is fixed at 0.05
    sigma = 0.05*torch.ones_like(mu)
    return
torch.distributions.MultivariateNormal(mu,
torch.diag_embed(sigma))
```

卷积-全连接混合的 Actor 网络

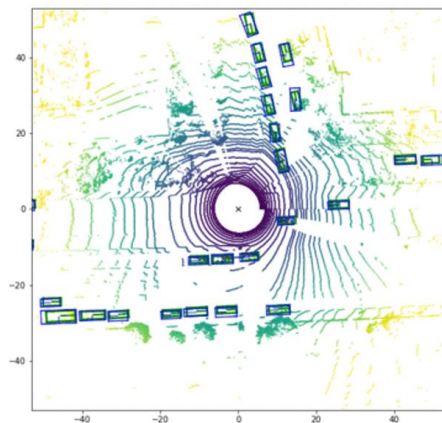
层级	计算/形状	作用
输入	(B, 5, 84, 84)	一批 5 通道 84×84 的俯视图
Conv1	16@20×20	8×8 卷积，步长 4，提取大感受野的局部特征
ReLU	同上	非线性激活，保留正响应
Conv2	32@9×9	4×4 卷积，步长 2，压缩空间尺寸并增加通道
ReLU	同上	再次非线性
Flatten	(B, 9×9×32=2592)	二维特征图拉成一维向量
FC1	256	全连接层，把 2592 维压缩到 256 维隐特征
ReLU	同上	继续非线性
FC2	2	输出 2 维均值 μ （转向 & 油门/刹车）
高斯分布	$N(\mu, \sigma^2 I)$	用固定方差 0.05^2 封装成连续动作分布

自动驾驶：价值网络(Critic)示例

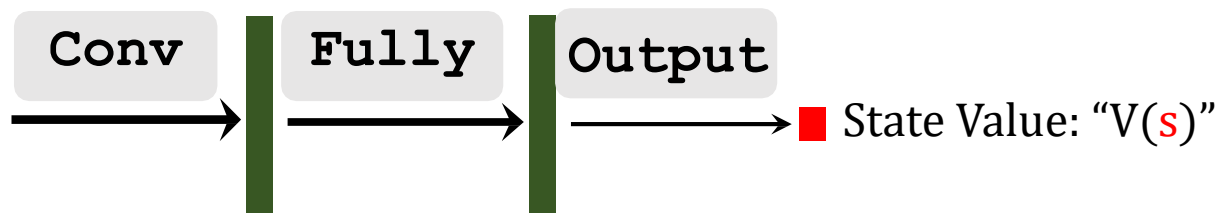
Value Network (Critic): $V_W(\mathbf{s})$

- Inputs: state $\mathbf{s}$.
- Output: state-value of the state $\mathbf{s}$.

state $\mathbf{s}$



84×84×5 多通道
鸟瞰图像



自动驾驶PPO算法 – Critic网络设计

```
class Critic(nn.Module):
    def __init__(self):
        super().__init__()
        # input is size 84x84x5
        # output is size 1
        self.conv1 = nn.Conv2d(5, 16,
kernel_size=8, stride=4)
        self.conv2 = nn.Conv2d(16, 32,
kernel_size=4, stride=2)
        self.fc1 = nn.Linear(9*9*32, 256)
        self.fc2 = nn.Linear(256, 1) # 256 -> 1

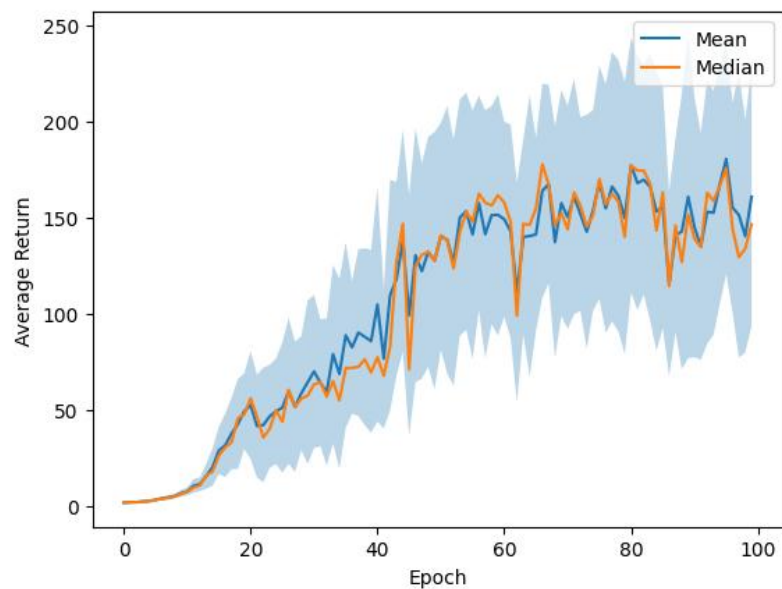
    def forward(self, x: torch.Tensor) ->
torch.Tensor:
    x = F.relu(self.conv1(x))
    x = F.relu(self.conv2(x))
    x = torch.flatten(x, start_dim=1)
    x = F.relu(self.fc1(x))
    x = self.fc2(x) # Bx256 -> Bx1
    output = torch.squeeze(x, dim=1)
    return output
```

卷积-全连接混合的 Critic 网络

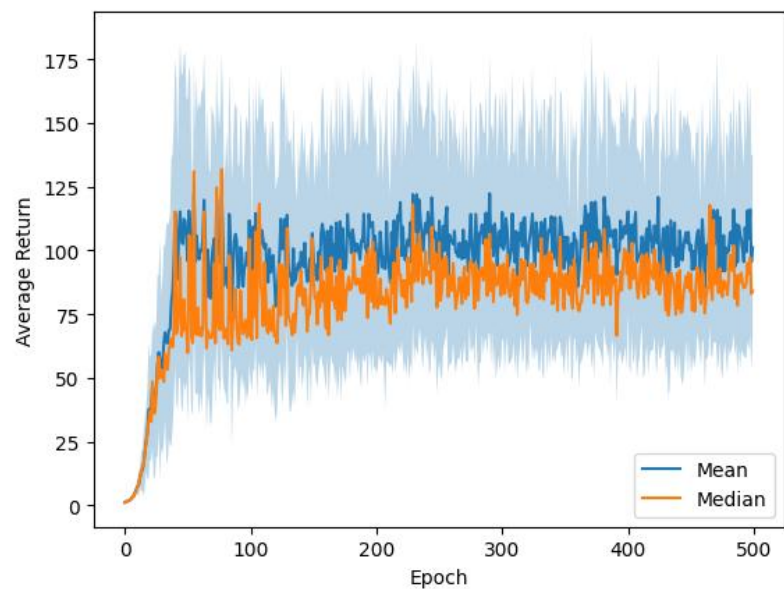
层级	计算/形状	作用
输入	(B, 5, 84, 84)	一批 5 通道 84×84 俯视图
Conv1	16@20×20	8×8 卷积，步长 4，提取高层空间特征
ReLU	同上	非线性激活
Conv2	32@9×9	4×4 卷积，步长 2，进一步压缩并增加通道
ReLU	同上	非线性
Flatten	(B, 2592)	把二维特征拉成一维
FC1	256	全连接隐藏层，提炼抽象特征
ReLU	同上	非线性
FC2	1	输出单值：当前状态的价值
Squeeze	(B,)	去掉最后一个维度，返回标量张量

PPO智能体训练回报

训练100epochs的平均回报

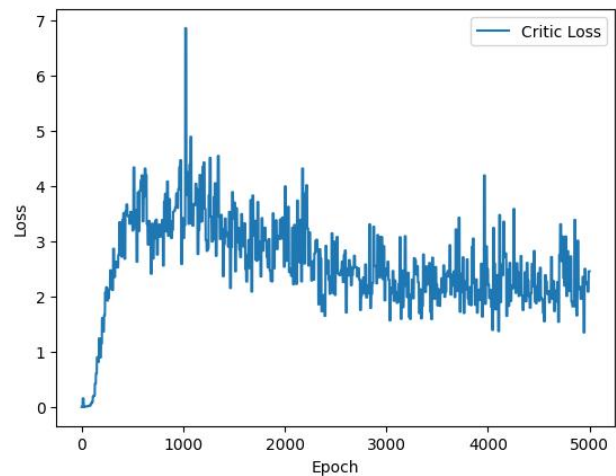
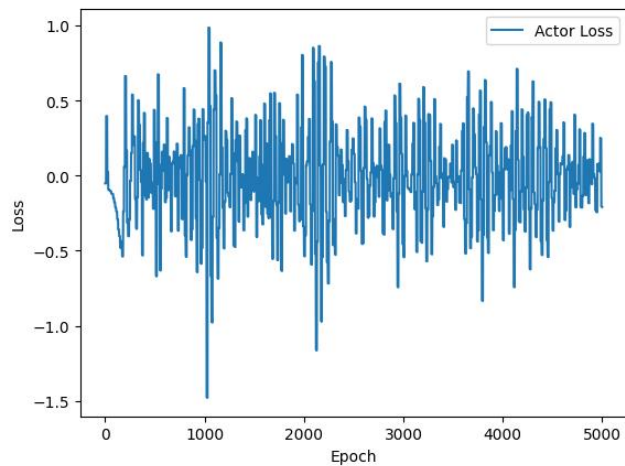
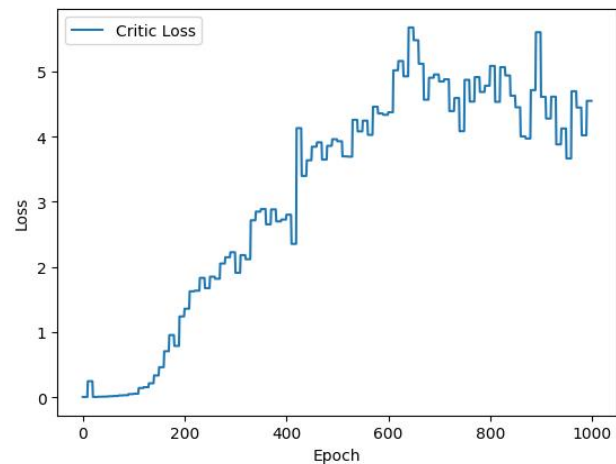
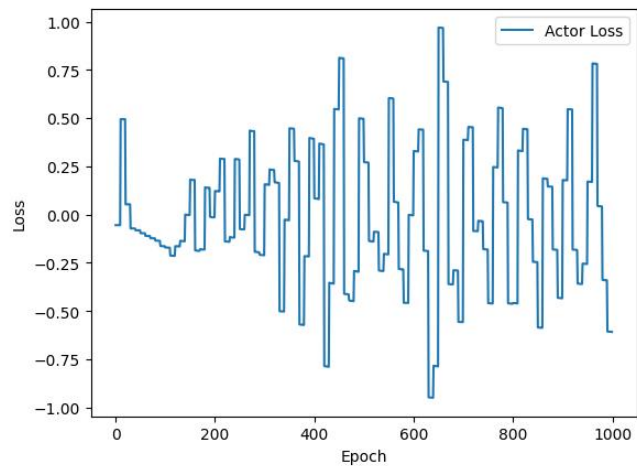


训练500epochs的平均回报



PPO智能体训练Loss性能曲线

Actor Loss (上: 100epochs,下: 500epochs) Critic Loss (上: 100epochs,下: 500epochs)



PP0决策控制效果展示环境配置

```
env = gym.make("MetaDrive-topdown",
config={
    "use_render": True,
    "image_observation": True,
    "horizon": 500,
    "num_scenarios": 1,
    "traffic_density": 0.1,
    "start_seed": 1000
})
```

智能体自动驾驶决策控制效果(训练500轮)



自动驾驶决策控制效果(专家级智能体)



作业三： 环境配置

```
env = gym.make("MetaDrive-topdown",
config={
    "use_render": True,
    "image_observation": True,
    "horizon": 500,
    "num_scenarios": 1,
    "traffic_density": 0.1,
    "start_seed": XXX (学号编号最后三位)
})
```

作业三

- 改进Actor网络和Critic网络设计
 - 提高训练效率
 - 提升自动驾驶决策控制性能
 - 作业提交：命名方式（学号+姓名：2117030116张三）
 1. Notebook文件(含成功运行结果展示)
 2. 展示视频文件
 3. 实验报告（提供模板？？）
-